Atualize suas planilhas do Google com os dados do SPO

Manual passo a passo — Sistema de Planejamento e Orçamento (SPO) · NGER/SEDUC-MT

Para que serve. Com a chave de acesso (API) que o NGER entrega para a sua equipe, a planilha do Google busca sozinha os relatórios do SPO (EMP, NOB, PED, EST EMP, FIP613 e PLAN 51). Você deixa de baixar o Excel à mão: a planilha se atualiza no horário que você escolher.

O que você precisa antes de começar

Item	Como obter
client_id	Código da sua chave, começa com `cli_`. Entregue pelo NGER.
client_secret	A "senha" da chave, começa com `sec_`. É mostrada uma única vez. **Guarde em local seguro e não compartilhe.**
Relatório liberado	O NGER informa quais relatórios a sua chave enxerga. Se pedir um não liberado, o sistema responde "Escopo insuficiente" (erro 403).
Conta Google	Qualquer conta que consiga criar uma planilha no Google Planilhas.

Visão geral — são 7 passos

1	Criar a planilha e a aba que vai receber os dados
2	Abrir o Apps Script (o "automatizador" do Google)
3	Colar o código pronto e escolher os relatórios (um ou vários)
4	Guardar a chave (client_id e client_secret) com segurança
5	Executar e autorizar pela primeira vez (aparece o menu **SPO**)
6	Acompanhar a cascata e conferir os dados
7	Agendar a verificação automática (e usar **SPO ➜ Atualizar agora** quando precisar)

Como funciona: os relatórios são baixados **em cascata**, um depois do outro: os pequenos primeiro (já dá para trabalhar neles) e a **EMP** e a **NOB**, que são grandes, por último e **em lotes**. Quando o tempo de uma execução do Google (6 minutos) está acabando, o script se agenda sozinho para continuar um minuto depois, sem você fazer nada. A verificação roda no horário que você agendar e só baixa o que o SPO atualizou.

As telas deste manual são ilustrações esquemáticas. O desenho do Google pode mudar um pouco de posição, mas os nomes dos botões e menus são os mesmos.

1 Criar a planilha e a aba

Abra **sheets.google.com** e crie uma **Planilha em branco**. Na parte de baixo, clique com o botão direito na aba "Página1", escolha **Renomear** e dê o nome que quiser (exemplo: `EST EMP`). Anote esse nome: ele vai no passo 3.



Ilustração 1 — a aba que vai receber os dados (aqui chamada "EST EMP").

2 Abrir o Apps Script

No menu da planilha, clique em **Extensões** e depois em **Apps Script**. Abre uma nova janela do navegador com o editor.



Ilustração 2 — Extensões → Apps Script.

3 Colar o código e escolher os relatórios

No editor, apague o código que já vem (`function myFunction() {}`), cole o código pronto da **última página deste manual** e clique no ícone de **disquete** (Salvar projeto).

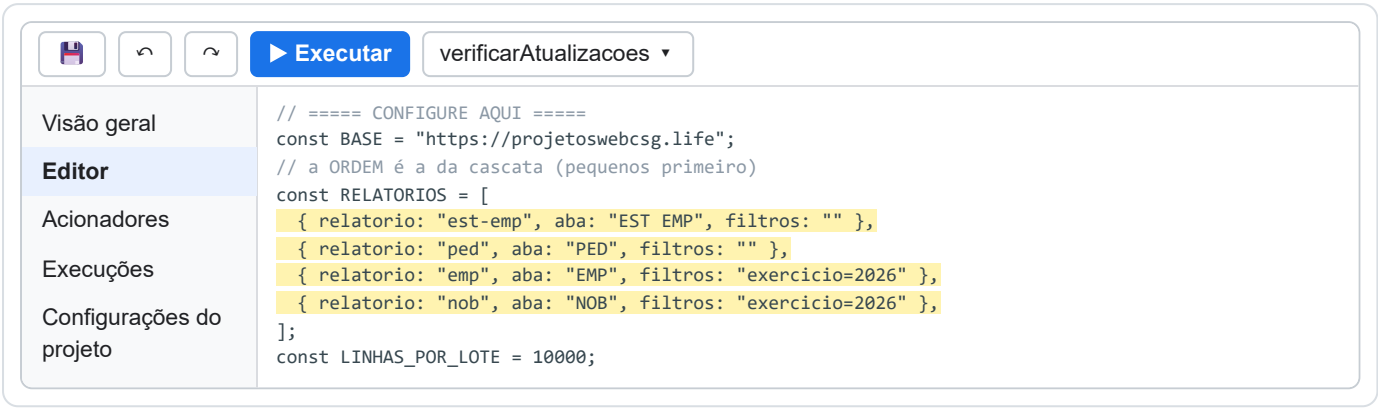


Ilustração 3 — você altera só a lista `RELATORIOS` : uma linha amarela por relatório, na ordem em que devem ser baixados.

Em cada linha	O que colocar
`relatorio`	`emp` , `nob` , `ped` , `est-emp` , `fip613` ou `plan51`
`aba`	Nome da aba que recebe os dados. Se não existir, o script cria. **Use um nome diferente em cada linha.**
`filtros`	Vazio traz tudo o que sua chave enxerga. Para limitar: `"exercicio=2026"` . Para mais de um filtro, junte com `&` : `"exercicio=2026&uo=26201"` . **Recomendado na EMP e na NOB.**
`LINHAS_POR_LOTE`	Quantas linhas baixar por vez (padrão 10000). Só mexa se aparecer erro de tempo com frequência: diminua para 5000.

4 Guardar a chave com segurança

Nunca escreva o `client_id` e o `client_secret` dentro do código. Guarde-os nas **Propriedades do script**:

① No menu da esquerda, clique em **Configurações do projeto** (ícone de engrenagem).
② Role até **Propriedades do script** e clique em **Adicionar propriedade de script**.
③ Crie duas propriedades, **com estes nomes exatos** (maiúsculas), e clique em **Salvar propriedades do script**.

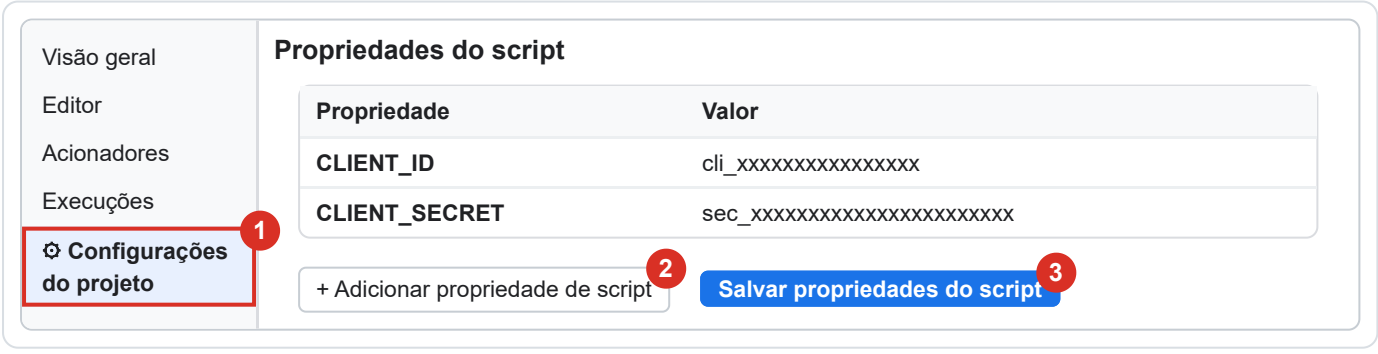


Ilustração 4 — as duas propriedades: `CLIENT_ID` e `CLIENT_SECRET` (valores de exemplo).

5 Executar e autorizar (só na primeira vez)

Volte ao **Editor**, selecione a função `verificarAtualizacoes` e clique em ▶ **Executar**. O Google vai pedir autorização, porque o script acessa a internet e a sua planilha:

① **Revisar permissões** → ② escolha a sua conta → ③ se aparecer "O Google não verificou este app", clique em **Avançado** e em **Acessar … (não seguro)** → ④ **Permitir**.

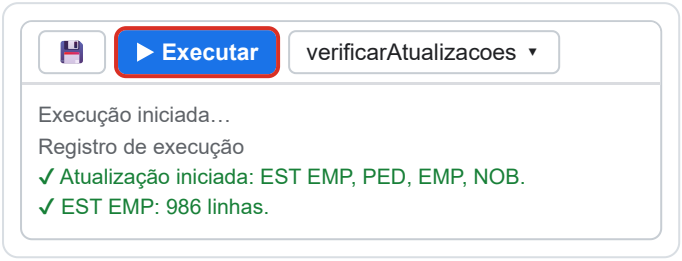


Ilustração 5a — Executar



Ilustração 5b — autorização

A autorização é pedida **uma vez**. Esta primeira execução baixa **tudo**, então a EMP e a NOB podem levar vários minutos e continuam sozinhas depois do limite de 6 minutos do Google. Se aparecer "Falha no login", confira os nomes e valores das propriedades do passo 4 (sem espaços sobrando). **Depois de colar o código, recarregue a planilha (F5):** aparece o menu **SPO** ao lado de Ajuda.

6 Acompanhar a cascata e conferir os dados

Volte para a planilha. As abas vão ficando prontas **uma a uma**, dos relatórios pequenos para os grandes, e você já pode trabalhar nas que terminaram enquanto a EMP e a NOB ainda carregam. Cada aba tem a **primeira linha como cabeçalho** (os mesmos nomes de coluna do Excel do SPO) e os dados logo abaixo. Para ver o andamento a qualquer momento, use **SPO → Ver andamento** (ilustração 8).

	A	B	C	D	E
1	**Exercício**	**Nº EST**	**Nº EMP**	**Empenho Atual**	**Valor EMP**
2	2026	1	2026NE000123	2026NE000123	10.500,00
3	2026	2	2026NE000456	2026NE000456	1.250,75
4	2026	3	2026NE000789	2026NE000789	87,40

EST EMP

Ilustração 6 — dados no padrão do Excel do SPO (exemplo com valores fictícios).

Os valores numéricos chegam como número, então você pode somar, filtrar e criar tabelas dinâmicas normalmente.

7 Agendar a verificação automática

① No menu da esquerda, clique em **Acionadores** (ícone de relógio) → ② **Adicionar acionador** (botão no canto inferior direito) → configure como abaixo → ③ **Salvar**.



Ilustração 7 — acionador de hora em hora. Use sempre a função `verificarAtualizacoes`.

Como funciona o agendamento: a cada hora o script só **pergunta ao SPO** se há carga nova em cada relatório (leva poucos segundos). Se nada mudou, termina sem baixar nada. Se o SPO atualizou algo (por exemplo, a EMP das 5h da manhã), só aquele relatório entra na fila e é baixado. Quer a planilha pronta logo cedo? Escolha um acionador diário às 6h e outro às 7h. O agendamento roda mesmo com a planilha fechada.

A cascata e o menu SPO

O que acontece quando há carga nova

1	**Verifica.** Consulta o SPO e monta a fila só com os relatórios que mudaram (ou todos, em *Refazer tudo*).
2	**Cascata.** Baixa um relatório de cada vez, na ordem da lista `RELATORIOS`. Quando um termina, o próximo começa. Os pequenos ficam prontos logo.
3	**Lotes.** A EMP e a NOB são baixadas em partes de 10 mil linhas, gravadas numa aba temporária **oculta** chamada, por exemplo, "NOB (novo)".
4	**Continua sozinho.** Perto dos 6 minutos do Google, o script guarda onde parou e se agenda para um minuto depois. Isso se repete até terminar. Você não precisa fazer nada.
5	**Troca no final.** Só quando o relatório inteiro chegou e foi conferido, os dados passam para a aba final e a temporária é apagada. Enquanto isso, a aba continua com os dados anteriores.
6	**Se o SPO receber carga nova no meio**, o script percebe e recomeça aquele relatório, para a planilha nunca ficar com dados misturados. Se um relatório falhar, os outros continuam e o erro aparece com o nome da aba.

O menu SPO

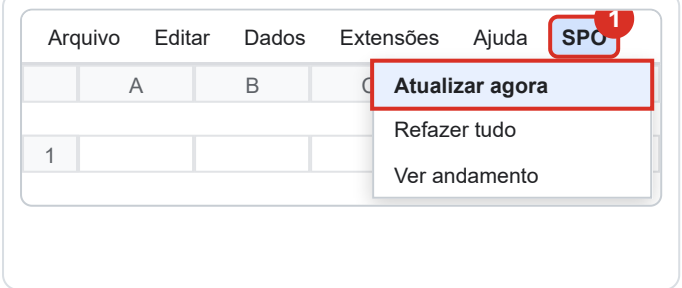


Ilustração 8a — menu SPO

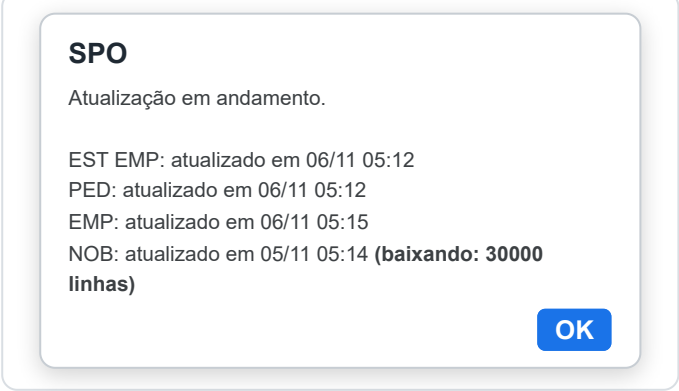


Ilustração 8b — Ver andamento

Menu SPO ➜	Para que serve
Atualizar agora	Faz a verificação na hora e baixa o que o SPO atualizou. Se está tudo em dia, avisa "Tudo já está atualizado".
Refazer tudo	Baixa todos os relatórios de novo, mesmo sem carga nova. Use se alguém mexeu nos dados de uma aba ou se algo ficou estranho.
Ver andamento	Mostra se há atualização em curso e quando cada aba foi atualizada pela última vez.

Cuidados

1. Cole o código no lugar do antigo, nunca embaixo dele. O Apps Script é um arquivo só: com o código duas vezes, o Google recusa (`Identifier 'BASE' has already been declared`). Para outro relatório, acrescente uma linha na lista.

2. No acionador, use só `verificarAtualizacoes` . Se você tinha um acionador de uma versão antiga (`atualizarDados` ou `atualizarTodos`), edite-o ou exclua-o. A função `continuarAtualizacao` o próprio script cria e apaga; não crie acionador para ela.

3. Limite de 10 milhões de células por planilha. Durante a carga, a EMP e a NOB ocupam o dobro (aba temporária + aba final). Use `exercicio=2026` nelas e evite várias abas muito grandes na mesma planilha. Se estourar, divida em duas planilhas.

4. Tempo diário do Google. Os agendamentos têm um limite de tempo somado por dia (cerca de 90 minutos em conta pessoal e 6 horas em Workspace; confirme na sua conta). A verificação de hora em hora gasta poucos segundos; só a carga da EMP e da NOB gasta minutos, e apenas quando o SPO atualiza.

5. A chave precisa ter o escopo de cada relatório da lista. Se faltar, só aquela aba falha (`403 Escopo insuficiente`). Peça ao NGER para liberar na mesma chave.

6. Não digite nada dentro das abas que recebem os dados (são reescritas a cada carga); faça cálculos em outra aba apontando para elas. Não apague nem renomeie as abas ocultas "(novo)" durante uma carga.

Consulta rápida

Relatórios e filtros úteis

`relatorio`	Conteúdo	Observação
`est-emp`	Estorno de empenho	Pequeno
`ped`	Pedidos (PED)	Pequeno
`fip613`	FIP 613	Pequeno. O ano é `ano=2026`
`plan51`	Créditos adicionais (PLAN 51)	Médio. Estado inteiro, sem filtro de UO
`emp`	Empenhos	Grande: baixado em lotes. **Use `exercicio=2026`**
`nob`	Notas de ordem bancária	Grande: baixado em lotes. **Use `exercicio=2026`**

Em `filtros`	**Efeito**
`""`	Tudo o que a chave enxerga. Sem indicar a UO, vêm só os dados da SEDUC.
`"exercicio=2026"`	Só o exercício de 2026 (vários: `exercicio=2025,2026`)
`"uo=26201"` / `"uo=all"`	Escolhe a UO / traz todas as UOs liberadas
`"exercicio=2026&uo=all"`	Combina filtros com `&`

Se algo der errado

Mensagem	O que fazer
`Identifier … has already been declared`	O código foi colado duas vezes. Apague tudo, cole uma só vez e, para outro relatório, acrescente uma linha na lista.
Falha no login … 401 “Credenciais invalidas”	Confira `CLIENT_ID` e `CLIENT_SECRET` no passo 4 (sem espaços, copiados inteiros). Se perdeu o secret, peça ao NGER para gerar outro.
… 403 “Escopo insuficiente”	Sua chave não tem esse relatório. Peça ao NGER para liberar (a chave é a mesma). Só a aba desse relatório falha.
… 403 “Acesso expirado” ou “revogado”	A chave venceu ou foi desativada. Peça renovação ao NGER.
“Exceeded maximum execution time”	Normal de vez em quando: a cascata continua sozinha em nova execução. Se for frequente, diminua `LINHAS_POR_LOTE` para 5000.
“…exceed the limit of 10000000 cells” (limite de células)	A planilha ficou grande demais. Use filtro de exercício na EMP e na NOB ou divida em duas planilhas.
A atualização “não termina”	Veja **SPO → Ver andamento**. Se estiver parada por mais de 3 horas, o próximo agendamento descarta e recomeça; ou use **SPO → Refazer tudo**.
Aba “(novo)” oculta sobrando	É a aba temporária de uma carga que falhou. Pode excluí-la; o script apaga e recria sozinho na próxima carga.
Dados numa aba nova que você não esperava	O script cria a aba com o nome indicado em `aba` se ela não existir. Confira a grafia.

Segurança. A chave dá acesso aos dados do SPO. Não envie o secret por e-mail ou chat, não o coloque em planilha compartilhada e quem sair da equipe deve ter a chave revogada (peça ao NGER).

Dúvidas, novos relatórios ou renovação de acesso: NGER/SEDUC-MT. Endereço do sistema: **https://projetoswebcsg.life**

Código pronto para colar (passo 3)

Copie tudo, cole no Apps Script (apagando o que já existe) e altere somente a lista `RELATORIOS` (e, se precisar, `LINHAS_POR_LOTE`). Para um único relatório, deixe uma linha só. O arquivo `google_planilhas_varios.gs`, entregue junto com este manual, tem o mesmo código para copiar sem erros de formatação do PDF.

```
// ============================================================
//  SPO -> Google Planilhas  (fila em cascata, carga em lotes)
// ============================================================

// ============ CONFIGURE AQUI ============
const BASE = "https://projetoswebcsg.life";

// A ORDEM É A DA CASCATA: os relatórios pequenos primeiro (ficam prontos logo)
// e os grandes por último. Um item por relatório.
//   relatorio: emp | nob | ped | est-emp | fip613 | plan51
//   aba:       nome da aba que recebe os dados (criada se não existir; um nome diferente em cada linha)
//   filtros:   "" = tudo que a chave enxerga | ex.: "exercicio=2026"
const RELATORIOS = [
  { relatorio: "est-emp", aba: "EST EMP", filtros: "" },
  { relatorio: "ped",     aba: "PED",     filtros: "" },
  { relatorio: "fip613",  aba: "FIP613",  filtros: "" },
  { relatorio: "plan51",  aba: "PLAN 51", filtros: "" },
  { relatorio: "emp",     aba: "EMP",     filtros: "exercicio=2026" },
  { relatorio: "nob",     aba: "NOB",     filtros: "exercicio=2026" },
];

// Linhas baixadas por vez. Se aparecer "Exceeded maximum execution time" com
// frequência, diminua (ex.: 5000). Não precisa passar de 20000.
const LINHAS_POR_LOTE = 10000;
// ========================================

const LIMITE_MS = 5 * 60 * 1000;       // o Google corta em 6 min: paramos antes e continuamos depois
const CICLO_VELHO_MS = 3 * 60 * 60 * 1000; // ciclo parado há mais de 3 h é descartado

// ---------- menu da planilha ----------
function onOpen() {
  SpreadsheetApp.getUi().createMenu("SPO")
    .addItem("Atualizar agora", "atualizarAgora")
    .addItem("Refazer tudo (baixar de novo)", "refazerTudo")
    .addItem("Ver andamento", "verAndamento")
    .addToUi();
}

// ---------- pontos de entrada ----------
// Agende ESTA função em Acionadores (ex.: de hora em hora): só baixa o que o SPO atualizou.
function verificarAtualizacoes() { iniciar_(false, false); }
// Menu SPO > Atualizar agora: faz a mesma verificação na hora.
function atualizarAgora() { iniciar_(false, true); }
// Menu SPO > Refazer tudo: baixa todos os relatórios de novo, mesmo sem carga nova.
function refazerTudo() { iniciar_(true, true); }

function verAndamento() {
  const p = PropertiesService.getScriptProperties();
  const ciclo = lerCiclo_();
  const linhas = RELATORIOS.map((c) => {
    const ts = p.getProperty("ULTIMA_" + c.aba);
    const quando = ts ? Utilities.formatDate(new Date(Number(ts)), Session.getScriptTimeZone(), "dd/MM HH:mm") : "nunca";
    let estado = "";
    if (ciclo) {
      const pos = ciclo.fila.findIndex((f) => RELATORIOS[f.i].aba === c.aba);
      if (pos === 0) estado = ciclo.fila[0].job ? "  (baixando: " + (ciclo.fila[0].job.escritas || 0) + " linhas)" : "  (baixando)";
      else if (pos > 0) estado = "  (na fila)";
    }
    return c.aba + ": atualizado em " + quando + estado;
  });
  const cab = ciclo ? "Atualização em andamento.\n\n" : "Nenhuma atualização em andamento.\n\n";
  SpreadsheetApp.getUi().alert("SPO", cab + linhas.join("\n"), SpreadsheetApp.getUi().ButtonSet.OK);
}

// ---------- 1) verifica o que mudou e monta a fila ----------
function iniciar_(forcar, avisar) {
  const lock = LockService.getScriptLock();
  if (!lock.tryLock(15000)) { aviso_("Já há uma atualização em andamento.", avisar); return; }
  let comecar = false;
  try {
    const atual = lerCiclo_();
    if (atual && Date.now() - atual.inicio < CICLO_VELHO_MS) {
      aviso_("Já há uma atualização em andamento (use SPO > Ver andamento).", avisar);
      return;
    }
    const p = PropertiesService.getScriptProperties();
```

```
    const h = login_();
    const fila = [];
    const erros = [];
    RELATORIOS.forEach((c, i) => {
      try {
        const carga = cargaAtual_(c, h);
        if (forcar || p.getProperty("CARGA_" + c.aba) !== carga) fila.push({ i: i, carga: carga, tentativas: 0 });
      } catch (e) {
        Logger.log("ERRO ao consultar " + c.aba + ": " + e.message);
        erros.push(c.aba + ": " + e.message);
      }
    });
    if (!fila.length) {
      aviso_(erros.length ? "Nada para baixar, com erros: " + erros.join(" | ") : "Tudo já está atualizado.", avisar);
      if (erros.length) throw new Error("Falha ao consultar -> " + erros.join(" | "));
      return;
    }
    gravarCiclo_({ inicio: Date.now(), fila: fila, erros: erros });
    aviso_("Atualização iniciada: " + fila.map((f) => RELATORIOS[f.i].aba).join(", ") + ".", avisar);
    comecar = true;
  } finally {
    lock.releaseLock();
  }
  if (comecar) continuarAtualizacao();
}

// ---------- 2) baixa em cascata e em lotes (continua sozinho em novas execuções) ----------
function continuarAtualizacao() {
  const lock = LockService.getScriptLock();
  if (!lock.tryLock(15000)) return; // outra execução em andamento
  try {
    apagarGatilhos_();
    const ciclo = lerCiclo_();
    if (!ciclo) return;
    const t0 = Date.now();
    try {
      const h = login_();
      while (ciclo.fila.length) {
        const item = ciclo.fila[0];
        const c = RELATORIOS[item.i];
        let r;
        try {
          r = passo_(c, item, h, t0);
        } catch (e) {
          Logger.log("ERRO em " + c.aba + ": " + e.message);
          ciclo.erros.push(c.aba + ": " + e.message);
          descartarTemporaria_(c);
          ciclo.fila.shift();
          gravarCiclo_(ciclo);
          continue;
        }
        if (r === "pronto") ciclo.fila.shift();
        gravarCiclo_(ciclo);
        if (r === "tempo") { agendar_(); return; }
      }
    } catch (e) {
      // erro inesperado (ex.: login): descarta o ciclo para a próxima verificação recomeçar
      PropertiesService.getScriptProperties().deleteProperty("CICLO");
      throw e;
    }
    PropertiesService.getScriptProperties().deleteProperty("CICLO");
    if (ciclo.erros.length) {
      aviso_("Concluído com erros: " + ciclo.erros.join(" | "), true);
      throw new Error("Alguns relatórios falharam -> " + ciclo.erros.join(" | "));
    }
    aviso_("Atualização concluída.", true);
    Logger.log("Atualização concluída.");
  } finally {
    lock.releaseLock();
  }
}

// Um passo do relatório da vez. Devolve "pronto", "tempo" (acabou o tempo desta execução) ou "continuar".
function passo_(c, item, h, t0) {
  const planilha = SpreadsheetApp.getActive();
  if (!item.job) item.job = novoJob_(c, planilha);
  const job = item.job;
  const p = PropertiesService.getScriptProperties();

  if (job.fase === "baixando") {
    const tmp = planilha.getSheetByName(job.tmp);
    if (!tmp) throw new Error("a aba temporária '" + job.tmp + "' foi removida durante a atualização");
    for (;;) {
      if (acabouTempo_(t0, job.ultimaMs)) return "tempo";
      const ini = Date.now();
      const q = (c.filtros ? c.filtros + "&" : "") + "limit=" + LINHAS_POR_LOTE + (job.after ? "&after_id=" + job.after : "");
      const resp = buscar_(BASE + "/api/v1/dados/" + c.relatorio + "/export?" + q, h);
```

```
      // a carga mudou no meio do caminho? (linha de outra carga, ou contagem diferente)
      const ups = cabecalho_(resp, "X-Upload-Id").split(",").filter(Boolean);
      if (!job.uploads) job.uploads = ups.join(",");
      const conhecidos = job.uploads.split(",");
      if (ups.some((u) => conhecidos.indexOf(u) < 0)) return reiniciar_(c, item, "o SPO recebeu carga nova durante a baixa", h);

      const dados = Utilities.parseCsv(resp.getContentText());
      const cab = dados.shift();
      if (!job.colunas) {
        job.colunas = cab.length;
        const total = parseInt(cabecalho_(resp, "X-Total"), 10);
        job.total = isNaN(total) ? null : total;
        garantirGrade_(tmp, 1, job.colunas);
        tmp.getRange(1, 1, 1, job.colunas).setValues([cab]);
      }
      if (dados.length) {
        const linhas = dados.map(converterLinha_);
        garantirGrade_(tmp, job.escritas + 1 + linhas.length, job.colunas);
        tmp.getRange(job.escritas + 2, 1, linhas.length, job.colunas).setValues(linhas);
        job.escritas += linhas.length;
      }
      job.after = cabecalho_(resp, "X-Next-After-Id");
      job.ultimaMs = Date.now() - ini;
      gravarItem_(item);
      if (!job.after || !dados.length) break;
    }
    if (job.total !== null && job.escritas !== job.total) return reiniciar_(c, item, "contagem difere (" + job.escritas + " x " +
job.total + ")", h);
    if (cargaAtual_(c, h) !== item.carga) return reiniciar_(c, item, "o SPO recebeu carga nova durante a baixa", h);
    job.fase = "trocando";
    job.copiadas = 0;
    gravarItem_(item);
  }

  if (job.fase === "trocando") {
    // só agora a aba final é substituída (de uma vez), para o usuário nunca vê-la pela metade por muito tempo
    const tmp = planilha.getSheetByName(job.tmp);
    if (!tmp) throw new Error("a aba temporária '" + job.tmp + "' foi removida durante a atualização");
    let dest = planilha.getSheetByName(c.aba);
    if (!dest) dest = planilha.insertSheet(c.aba);
    const total = job.escritas + 1; // com o cabeçalho
    if (job.copiadas === 0) dest.clearContents();
    garantirGrade_(dest, total, job.colunas);
    while (job.copiadas < total) {
      if (acabouTempo_(t0, job.ultimaMs)) return "tempo";
      const ini = Date.now();
      const n = Math.min(20000, total - job.copiadas);
      tmp.getRange(job.copiadas + 1, 1, n, job.colunas)
        .copyTo(dest.getRange(job.copiadas + 1, 1), SpreadsheetApp.CopyPasteType.PASTE_VALUES, false);
      job.copiadas += n;
      job.ultimaMs = Date.now() - ini;
      gravarItem_(item);
    }
    planilha.deleteSheet(tmp);
    p.setProperty("CARGA_" + c.aba, item.carga);
    p.setProperty("ULTIMA_" + c.aba, String(Date.now()));
    Logger.log(c.aba + ": " + job.escritas + " linhas.");
    return "pronto";
  }
  throw new Error("estado desconhecido: " + job.fase);
}

function reiniciar_(c, item, motivo, h) {
  item.tentativas = (item.tentativas || 0) + 1;
  Logger.log(c.aba + ": " + motivo + " -> recomeçando (tentativa " + item.tentativas + ").");
  descartarTemporaria_(c);
  if (item.tentativas > 3) throw new Error(motivo + "; desisti após 3 tentativas");
  item.job = null;
  item.carga = cargaAtual_(c, h);
  gravarItem_(item);
  return "continuar";
}

// ---------- utilidades ----------
function login_() {
  const p = PropertiesService.getScriptProperties();
  const r = UrlFetchApp.fetch(BASE + "/api/auth/token", {
    method: "post", contentType: "application/json", muteHttpExceptions: true,
    payload: JSON.stringify({ client_id: p.getProperty("CLIENT_ID"), client_secret: p.getProperty("CLIENT_SECRET") }),
  });
  if (r.getResponseCode() !== 200) throw new Error("Falha no login: " + r.getContentText());
  return { headers: { Authorization: "Bearer " + JSON.parse(r.getContentText()).access_token }, muteHttpExceptions: true };
}

// Identifica a carga do SPO + o filtro: se não mudou, não há o que baixar.
function cargaAtual_(c, h) {
  const meta = buscar_(BASE + "/api/v1/dados/" + c.relatorio + "/meta" + (c.filtros ? "?" + c.filtros : ""), h);
  return JSON.parse(meta.getContentText()).upload_id.join(",") + "|" + c.filtros;
```

```
}

// Uma nova tentativa em caso de falha de rede ou erro 5xx.
function buscar_(url, h) {
  let ultimo = "";
  for (let t = 1; t <= 2; t++) {
    try {
      const r = UrlFetchApp.fetch(url, h);
      const cod = r.getResponseCode();
      if (cod === 200) return r;
      ultimo = cod + " " + String(r.getContentText()).slice(0, 200);
      if (cod < 500) break; // 4xx: repetir não adianta
    } catch (e) {
      ultimo = e.message;
    }
    if (t === 1) Utilities.sleep(3000);
  }
  throw new Error(ultimo);
}

function cabecalho_(resp, nome) {
  const hs = resp.getHeaders();
  const k = Object.keys(hs).filter((x) => x.toLowerCase() === nome.toLowerCase())[0];
  return k ? String(hs[k]) : "";
}

// Números chegam como texto com ponto decimal; vira número, exceto códigos com zero à esquerda ou muito longos.
function converterLinha_(linha) {
  return linha.map((v) => (/^-?(0|[1-9]\d{0,14})(\.\d+)?$/.test(v) ? Number(v) : v));
}

function acabouTempo_(t0, ultimaMs) {
  const gasto = Date.now() - t0;
  if (gasto < 10000) return false; // sempre dá ao menos um passo por execução
  return gasto + 1.5 * (ultimaMs || 0) > LIMITE_MS;
}

function garantirGrade_(aba, linhas, colunas) {
  if (aba.getMaxRows() < linhas) aba.insertRowsAfter(aba.getMaxRows(), linhas - aba.getMaxRows());
  if (aba.getMaxColumns() < colunas) aba.insertColumnsAfter(aba.getMaxColumns(), colunas - aba.getMaxColumns());
}

function novoJob_(c, planilha) {
  const nome = c.aba + " (novo)";
  const velha = planilha.getSheetByName(nome);
  if (velha) planilha.deleteSheet(velha);
  const tmp = planilha.insertSheet(nome);
  tmp.hideSheet();
  return { fase: "baixando", tmp: nome, after: "", escritas: 0, total: null, uploads: "", colunas: 0, ultimaMs: 0, copiadas: 0 };
}

function descartarTemporaria_(c) {
  try {
    const planilha = SpreadsheetApp.getActive();
    const tmp = planilha.getSheetByName(c.aba + " (novo)");
    if (tmp) planilha.deleteSheet(tmp);
  } catch (e) { Logger.log("não consegui apagar a aba temporária de " + c.aba + ": " + e.message); }
}

function lerCiclo_() {
  const t = PropertiesService.getScriptProperties().getProperty("CICLO");
  return t ? JSON.parse(t) : null;
}
function gravarCiclo_(ciclo) { PropertiesService.getScriptProperties().setProperty("CICLO", JSON.stringify(ciclo)); }
// grava o andamento do item da vez sem perder o resto do ciclo
function gravarItem_(item) {
  const ciclo = lerCiclo_();
  if (ciclo && ciclo.fila.length) { ciclo.fila[0] = item; gravarCiclo_(ciclo); }
}

function agendar_() { ScriptApp.newTrigger("continuarAtualizacao").timeBased().after(60 * 1000).create(); }
function apagarGatilhos_() {
  ScriptApp.getProjectTriggers().forEach((g) => {
    if (g.getHandlerFunction() === "continuarAtualizacao") ScriptApp.deleteTrigger(g);
  });
}

function aviso_(msg, avisar) {
  Logger.log(msg);
  if (!avisar) return;
  try { SpreadsheetApp.getActive().toast(msg, "SPO", 10); } catch (e) { /* sem tela ativa */ }
}
```